I / O Streams

Module-1



Dr.M. Sivakumar
Professor & Technical Trainer / CSE
Saveetha School of Engineering, SIMATS
sivakumarm.sse@saveetha.com
+91-9500600868

Stream

- The **java.io** package contains all the classes required for input-output operations.
- All streams represent an input source and an output destination.
- The stream in the java.io package supports all the datatype including primitive.
- A stream can be defined as a sequence of data.
- There are two kinds of Streams
  - **Byte Stream**
  - **Character Stream**

Byte Streams

- Programs use byte streams to perform input and output of 8-bit bytes.
- All byte stream classes are descended from InputStream and OutputStream.
- It is suitable for any kind of file, however not quite appropriate for text files

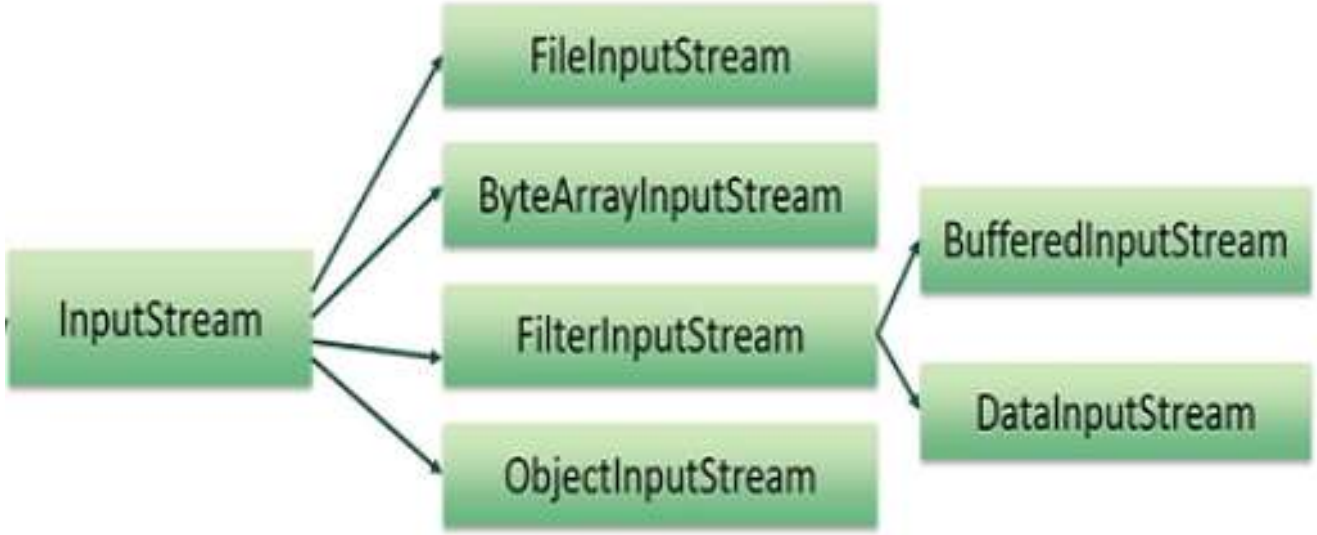
In Java, Byte Streams are again classified into the following two types:

1. **InputStream**
2. **OutputStream**

Byte Streams

1.Input Stream

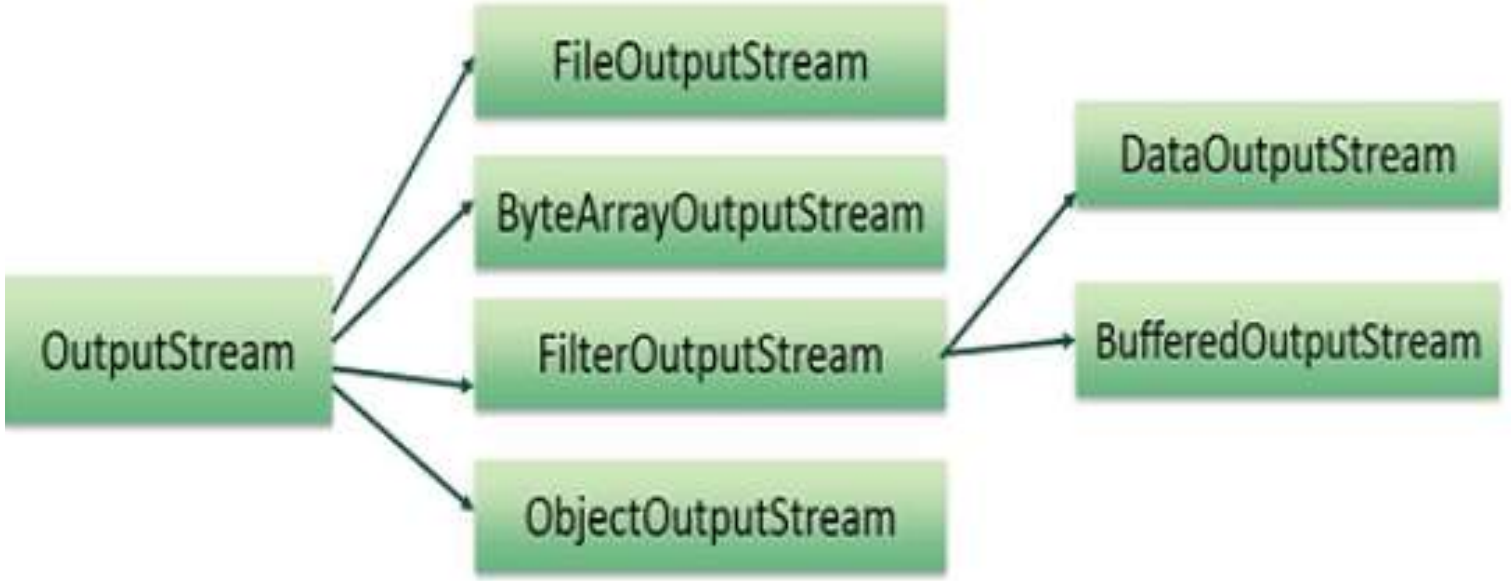
- These are used to read byte data from various input devices.
- InputStream is an abstract class and it is the super class of all the input byte streams.
- It provides an internal buffer for the bytes read from input stream



Byte Streams

2.Output Stream

- These are used to write byte data to various output devices.
- Output Stream is an abstract class and it is the superclass for all the output byte streams.
- It uses byte array to write data. The buffer automatically grows as data is written to it.



Byte Streams

1.Input Stream & Output Stream– Example

```
byte bWrite[] = { 11, 21, 3, 40, 5 };
OutputStream os = new FileOutputStream ("test.txt");
for (int x = 0; x < bWrite.length; x++)
{
os.write (bWrite[x]); // writes the bytes
}
os.close ();
InputStream is = new FileInputStream ("test.txt");
int size = is.available ();
for (int i = 0; i < size; i++)
{
System.out.print ((char) is.read () + " ");
}
is.close ();
```

FileOutputStream - Example

```
class FileOutDemo {
    public static void main(String args[]) {
        try {
            FileOutputStream fout = new FileOutputStream("abc.txt");
            String s = "Sourav Ganguly is my favorite player";
            byte b[] = s.getBytes();
            fout.write(b);
            fout.close();
            System.out.println("Success...");
        } catch (Exception e) {
                System.out.println(e);
        }
    }
}
```

Example (FileInputStream)

```
class SimpleRead {
    public static void main(String args[]) {
        try {
            FileInputStream fin = new FileInputStream("abc.txt");
            int i = 0;
            while ((i = fin.read()) != -1) {
                System.out.println((char) i);
            }
            fin.close();
        } catch (Exception e) {
                 System.out.println(e);
        }
    }
}
```

Example of Byte Steams

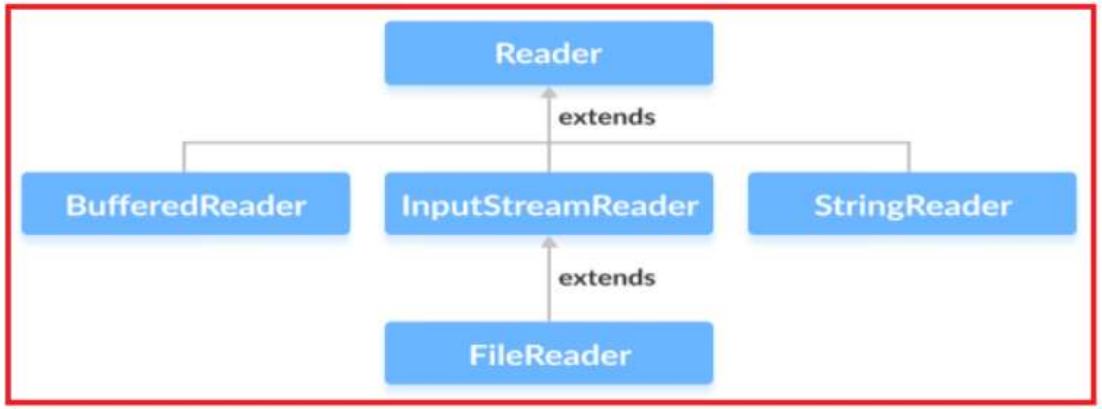
```
import java.io.*;
public class CopyFile {
    public static void main(String args[]) throws IOException {
        FileInputStream in = null;
        FileOutputStream out = null;
        try {
            in = new FileInputStream("input.txt");
            out = new FileOutputStream("output.txt");
            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

Character Streams

- A **character stream** will read a file character by character.
- Character Stream is a higher level concept than **Byte Stream.**
- It is also defined by using two abstract class at the top of hierarchy, they are Reader and Writer.

1.Reader Character Stream

- The reader is the super most class of all classes which are used for performing reading operations from any input device char by char.



Character Streams

1.Writer Character Stream

- The writer is the super most class of all classes that are used for performing writing operations from any output device char by char.
- The Writer class of the java.io package is an abstract superclass that represents a stream of characters.
- Since Writer is an abstract class, it is not useful by itself. However, its subclasses can be used to write data.



Character Streams

Reader/Writer Character Stream - Example

```
File file = new File("E:/TextFile1.txt");
FileReader reader = new FileReader(file);
char chars[] = new char[(int) file.length()];
//Reading data from the file
reader.read(chars);
//Writing data to another file
File out = new File("E:/TextFile2.txt");
FileWriter writer = new FileWriter(out);
//Writing data to the file
writer.write(chars);
writer.flush();
```

Character Streams

Reader/Writer Character Stream - Example

```
String text;
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
text = br.readLine();    //Reading String
System.out.println(text);
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

int a = Integer.parseInt(br.readLine());
System.out.println(a);
```

Reader

- The Java `Reader` class is the base class of all Reader's in the I-O API.
- Subclasses include a `FileReader, FileWriter, BufferedReader, BufferedWriter, InputStreamReader, StringReader` and several others.
- Here is a simple Java IO Reader example:

```
Reader reader = new FileReader("c:\\data\\myfile.txt");
int data = reader.read();
while (data != -1) {
    char dataChar = (char) data;
    data = reader.read();
}
```

- Combining Readers with InputStream

```
Reader reader = new InputStreamReader("c:\\data\\myfile.txt");
```

Writer

- The Java `Writer` class is the base class of all Writers in the I-O API.
- Subclasses include `BufferedWriter`, `PrintWriter`, `StringWriter` and several others.
- Here is a simple Java IO Writer example:

```
Writer writer = new FileWriter("c:\\data\\file-output.txt");
writer.write("Hello World Writer");
writer.close();
```

- Combining Readers With OutputStreams

```
Writer writer =
        new OutputStreamWriter("c:\\data\\file-output.txt");
```

FileWriter

- FileWriter is useful to create a file writing characters into it.
- This class inherits from the OutputStreamWriter class.
- Constructors of FileWriter class are as follows:

Sr.	Constructor
1	**FileWriter(File file)** Constructs a FileWriter object given a File object.
2	**FileWriter (File file, boolean append)** Constructs a FileWriter object given a File object, it will append if second parameter is true.
3	**FileWriter(String file)** Constructs a FileWriter object from the path given in parameter.
4	**FileWriter (String file, boolean append)** Constructs a FileWriter object from the path given, it will append if second parameter is true.

FileWriter – Methods

Sr.	Methods
1	**public void write (int c) throws IOException** Writes a single character.
2	**public void write (char [] str) throws IOException** Writes an array of characters.
3	**public void write(String str)throws IOException** Writes a string
4	**public void write(String str,int off,int len)throws IOException** Writes a portion of a string. Here off is offset from which to start writing characters and len is number of character to write.

FileReader

- FileReader is useful to read data in the form of characters from a text file.
- This class inherit from the InputStreamReader Class.
- Constructors of FileReader class are as follows:

Sr.	Constructor
1	**FileReader(File file)** Creates a FileReader , given the File to read from.
2	**FileReader(String fileName)** Creates a new FileReader , given the name of the file to read from.
3	**FileReader(FileDescripter fd)** Creates a new FileReader , given the FileDescripter to read from.

FileReader (Cont.)

Sr.	Methods
1	**public int read () throws IOException** Reads a single character. This method will block until a character is available, an I/O error occurs, or the end of the stream is reached.
2	**public int read(char[] cbuff) throws IOException** Reads characters into an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.
3	**public abstract int read(char[] buff, int off, int len) throws IOException** Reads characters into a portion of an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached. Parameters: cbuf – Destination buffer off – Offset at which to start storing characters len – Maximum number of characters to read
4	**public long skip(long n) throws IOException** Skips characters. This method will block until some characters are available, an I/O error occurs, or the end of the stream is reached.

Example FileReader/FileWriter

```
import java.io.*;

public class CopyFile {
    public static void main(String args[]) throws IOException {
        FileReader in = null;
        FileWriter out = null;
        try {
            in = new FileReader("input.txt");
            out = new FileWriter("output.txt");
            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

BufferedReader

- The `java.io.BufferedReader` class reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines.
- Following are the important points about `BufferedReader`:
  - The buffer size may be specified, or the default size may be used.
  - Each read request made of a Reader causes a corresponding read request to be made of the underlying character or byte stream.

Sr.	Constructor
1	**BufferedReader(Reader in)** This creates a buffering character-input stream that uses a default-sized input buffer.
2	**BufferedReader(Reader in, int sz)** This creates a buffering character-input stream that uses an input buffer of the specified size.

BufferedReader – Example

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

class BufferedReaderDemo {
    public static void main(String[] args) throws IOException {
        FileReader fr = new FileReader("input.txt");
        BufferedReader br = new BufferedReader(fr);
        char c[] = new char[20];
        br.skip(8);
        if (br.ready()) {
            System.out.println(br.readLine());
            br.read(c);
            for (int i = 0; i < 20; i++) {
                System.out.print(c[i]);
            }
        }
    }
}
```

```
G:\Darshan\Java 2019\PPTs
ava.io.BufferedReader;
import java.io.FileR
```

BufferedReader (Cont.)

Sr.	Methods
1	**void close()** This method closes the stream and releases any system resources associated with it.
2	**int read()** This method reads a single character.
3	**int read(char[] cbuf, int off, int len)** This method reads characters into a portion of an array.
4	**String readLine()** This method reads a line of text.
5	**void reset()** This method resets the stream.
6	**long skip(long n)** This method skips characters.

File class

- Java `File` class represents the files and directory pathnames in an **abstract manner**. This class is used for creation of **files** and **directories**, **file searching**, **file deletion** etc.
- The File object represents the actual file/directory on the disk. Below given is the list of constructors to create a File object.

Sr.	Constructor
1	**File(String pathname)** Creates a new File instance by converting the given pathname string into an abstract pathname.
2	**File(String parent, String child)** Creates a new File instance from a parent pathname string and a child pathname string.
3	**File(URI uri)** Creates a new File instance by converting the given file: URI into an abstract pathname.

```java
import java.io.File;
class FileDemo {
    public static void main(String args[]) {
        File f1 = new File("FileDemo.java");
        System.out.println("File Name: " + f1.getName());
        System.out.println("Path: " + f1.getPath());
        System.out.println("Abs Path: " + f1.getAbsolutePath());
        System.out.println("Parent: " + f1.getParent());
        System.out.println(f1.exists() ? "exists" : "does not exist");
        System.out.println(f1.canWrite() ? "is writeable" : "is not writeable");
        System.out.println (f1.canRead () ? "is readable" : "is not readable");
        System.out.println ("is " + (f1.isDirectory() ? "" : "not" + " a directory"));
        System.out.println(f1.isFile() ? "is normal file" : "might be a named pipe");
        System.out.println(f1.isAbsolute() ? "is absolute" : "is not absolute");
        System.out.println("File last modified: " + f1.lastModified());
        System.out.println("File size: " + f1.length() + " Bytes");
    }
}
```

```
G:\Darshan\Java 2019\PPTs\HAD\Programs>java FileDemo
File Name: FileDemo.java
Path: FileDemo.java
Abs Path: G:\Darshan\Java 2019\PPTs\HAD\Programs\FileDemo.java
Parent: null
exists
is writeable
is readable
is not a directory
is normal file
is not absolute
File last modified: 1587202242054
File size: 951 Bytes
```

Methods of File Class

Sr.	Method
1	**public boolean isAbsolute()** Tests whether this abstract pathname is absolute. Returns true if this abstract pathname is absolute, false otherwise
2	**public String getAbsolutePath()** Returns the absolute pathname string of this abstract pathname.
3	**public boolean canRead()** Tests whether the application can read the file denoted by this abstract pathname. Returns true if and only if the file specified by this abstract pathname exists and can be read by the application; false otherwise.
4	**public boolean canWrite()** Tests whether the application can modify to the file denoted by this abstract pathname. Returns true if and only if the file system actually contains a file denoted by this abstract pathname and the application is allowed to write to the file; false otherwise.
5	**public boolean exists()** Tests whether the file or directory denoted by this abstract pathname exists. Returns true if and only if the file or directory denoted by this abstract pathname exists; false otherwise

Methods of File Class (Cont.)

Sr.	Method
6	**public boolean isDirectory()** Tests whether the file denoted by this abstract pathname is a directory. Returns true if and only if the file denoted by this abstract pathname exists and is a directory; false otherwise.
7	**public boolean isFile()** Tests whether the file denoted by this abstract pathname is a normal file. A file is normal if it is not a directory and, in addition, satisfies other system-dependent criteria
8	**public long lastModified()** Returns the time that the file denoted by this abstract pathname was last modified. Returns a long value representing the time the file was last modified, measured in milliseconds since the epoch (00:00:00 GMT, January 1, 1970).
9	**public long length()** Returns the length of the file denoted by this abstract pathname.
10	**public boolean delete()** Deletes the file or directory.
11	**public String[] list()** Returns an array of strings naming the files and directories in the directory denoted by this abstract pathname.